Software Evolution & Refactoring

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

24 April 2026

Today's Agenda

Agenda

The Journey So Far

Over the past 11 weeks, we have covered the journey of **building** software:

- **Weeks 1-3:** Agile Planning, Requirements, and UML Models.
- **Weeks 4-6:** System Architecture, Design Patterns, and HCI.
- **Weeks 7-9:** Containerisation (Docker) and Automated Testing.

You have built your Ground Control Station. Your tests are passing. Your containers are running. Before we look at what happens *next*, we need to formally name the methodology you have been using for the past month: **DevOps**.

The Traditional ”Wall of Conflict”

Historically, software companies had two entirely separate teams with conflicting goals:

Development (Dev)		Operations (Ops)
Goal: Change & Agility. *”We want to ship new features to users as fast as possible”*	**VS**	**Goal:** Stability & Uptime. *”Every time you give us new code, the servers crash. Stop changing things”*

The Result: Developers would write code, throw it over the ”Wall of Conflict” to Operations, and wash their hands of it. Deployments took months and frequently failed.

What is DevOps?

Definition

DevOps is the combination of cultural philosophies, practices, and tools that increases an organization's ability to deliver applications at high velocity. Focus: **Collaboration** between Dev and Ops, and **Automation** of the software delivery process.

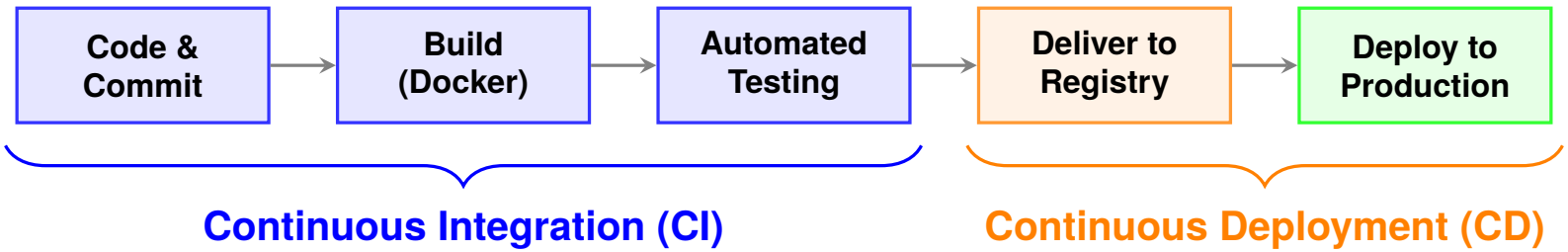
It removes the wall. The engineers who write the code are also responsible for deploying and maintaining the code.

How you already applied this:

- You acted as **Devs** when you wrote your FastAPI/Node.js endpoints.
- You acted as **Ops** when you wrote your `Dockerfile` and `docker-compose.yml` to define the server infrastructure as code!

Formalizing CI / CD

The primary engine of DevOps is the CI/CD Pipeline. You built the first half of this in Week 8 using GitHub Actions.



- **CI (Continuous Integration):** Automatically merging, building, and testing code to ensure the ‘main‘ branch is never broken.
- **CD (Continuous Deployment):** Automatically releasing that tested code directly to the live servers without human intervention.

The Consequence of High Velocity

By using DevOps and CI/CD, companies like Netflix and Amazon can deploy new code to production thousands of times a day.

The New Problem

If you can write, merge, and ship code instantly, the codebase grows and changes at an unprecedented rate. How do you prevent it from turning into a chaotic, unmaintainable mess?

This brings us to the core topic of today's lecture: **Software Evolution and Technical Debt.**

Today's Learning Objectives

By the end of this lecture, you should be able to:

1. **Understand** the principles of Software Evolution and Lehman's Laws.
2. **Define** Technical Debt and analyze its impact on software lifecycles.
3. **Identify** common "Code Smells" that indicate poor design.
4. **Apply** Refactoring techniques to improve code quality without altering external behavior.

Agenda

The Reality of Software

Software is not a physical bridge. When a bridge is built, it remains static until it decays. Software, however, is a living entity.

Software Evolution

The process of developing, maintaining, and updating software *after* its initial release to accommodate changing user requirements, hardware updates, or business goals.

Why does software evolve?

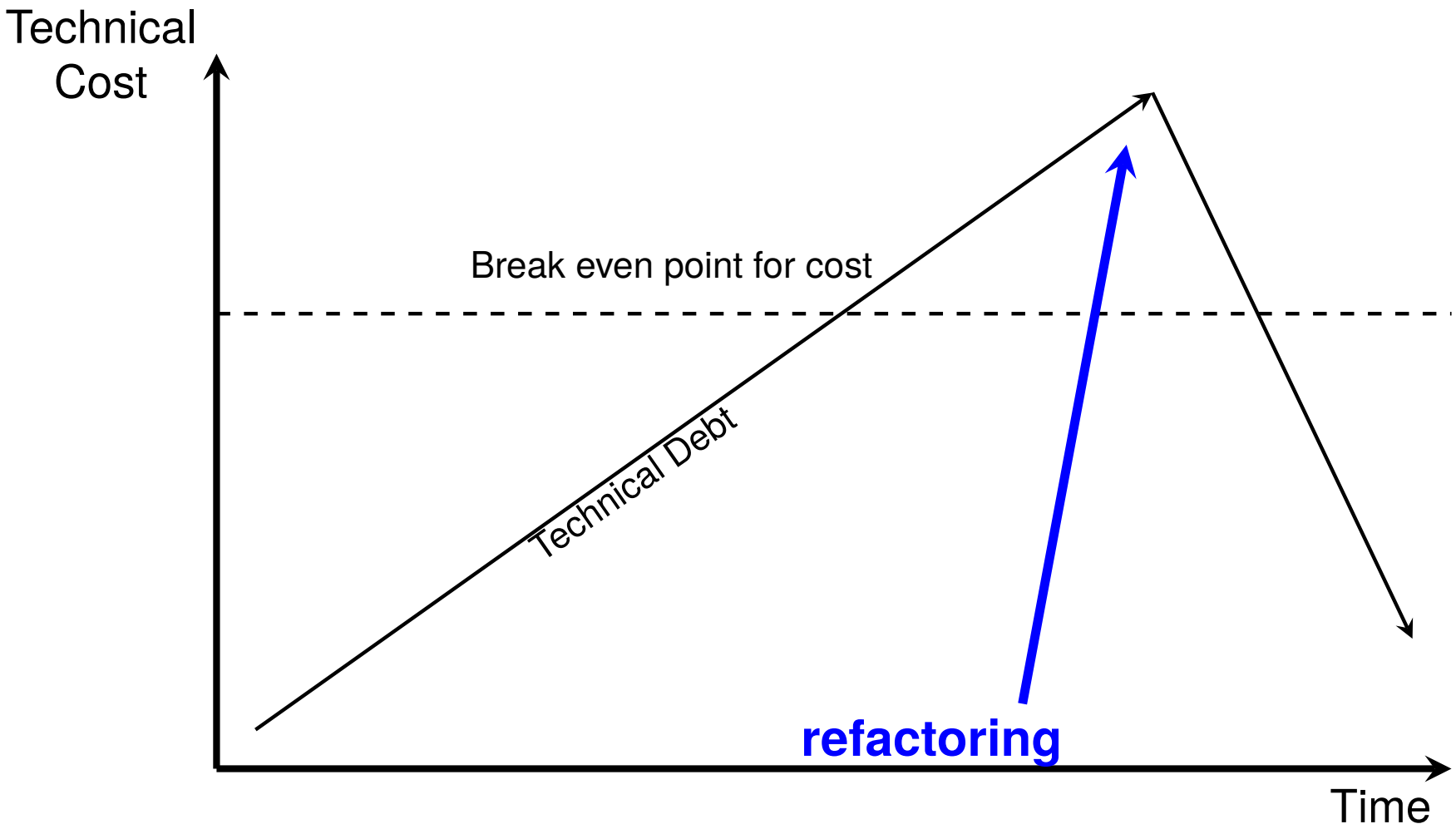
- The business environment changes (e.g., new robotics regulations).
- Underlying platforms change (e.g., Python 2 is deprecated for Python 3).
- Users discover new ways to use the system that developers never anticipated.

Lehman's Laws of Software Evolution

In the 1970s, Meir Lehman proposed a set of laws regarding software evolution. They remain universally true today.

1. **The Law of Continuing Change:** A system must continually adapt to its changing environment, or it will become progressively less useful until it dies.
2. **The Law of Increasing Complexity:** As a system evolves, its complexity increases unless explicit work is done to maintain or reduce it.
3. **The Law of Declining Quality:** The quality of a system will appear to decline over time unless it is rigorously maintained and adapted to changes in its operational environment.

Technical Debt and Refactoring



Agenda

What is Technical Debt?

As complexity increases (Lehman's 2nd Law), teams often take shortcuts to deliver features faster. This introduces **Technical Debt**.

The Financial Metaphor (Ward Cunningham)

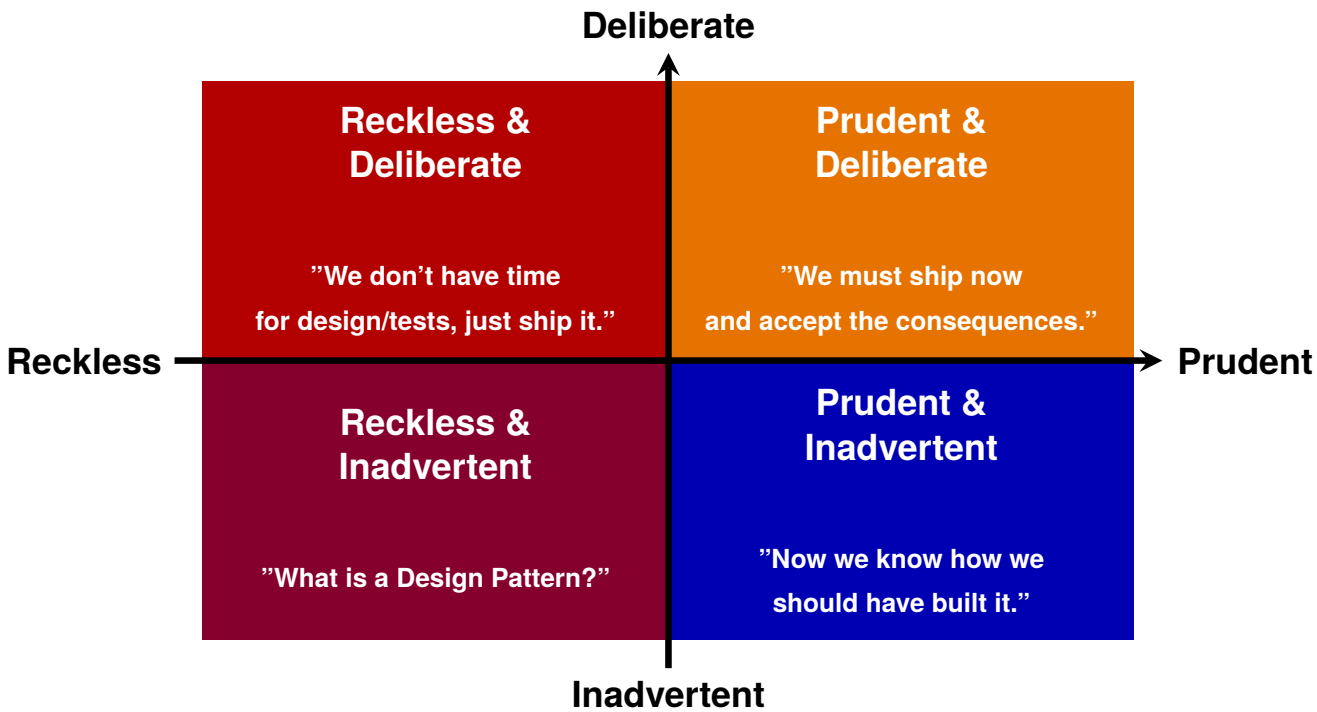
Shipping first-time code is like going into debt. A little debt speeds development so long as it is paid back promptly with a rewrite. The danger occurs when the debt is not repaid. Every minute spent on not-quite-right code counts as **interest** on that debt.

How do we pay the "interest"?

- Slower development times for future features.
- Increased bug rates.
- Developer burnout and frustration.

The Technical Debt Quadrant (Martin Fowler)

Not all technical debt is bad. Sometimes it is a strategic business decision.



Knowledge Check: Classify the Debt

Scenario: Your team is building the Ground Control Station. The deadline is tomorrow. You realize your database queries are unoptimized and might slow down if 1,000 robots connect. You decide to ship the code as-is to meet the deadline, but you immediately create a GitHub issue to rewrite the queries next sprint.

Question: Which quadrant of Technical Debt is this?

1. A) Reckless & Deliberate
2. B) Prudent & Inadvertent
3. C) Prudent & Deliberate
4. D) Reckless & Inadvertent

Knowledge Check: Classify the Debt

Scenario: Your team is building the Ground Control Station. The deadline is tomorrow. You realize your database queries are unoptimized and might slow down if 1,000 robots connect. You decide to ship the code as-is to meet the deadline, but you immediately create a GitHub issue to rewrite the queries next sprint.

Question: Which quadrant of Technical Debt is this?

1. A) Reckless & Deliberate
2. B) Prudent & Inadvertent
3. C) Prudent & Deliberate
4. D) Reckless & Inadvertent

Answer: C (Prudent & Deliberate)

You knew exactly what the consequences were (Deliberate), and you made a calculated business decision while planning for

Agenda

Identifying Debt: Code Smells

How do you know if your codebase has technical debt? You look for **Code Smells**.

A code smell is a surface indication that usually corresponds to a deeper problem in the system. It is not necessarily a bug—the code might work perfectly—but it indicates weaknesses in design that increase the risk of future bugs.

Common Code Smells

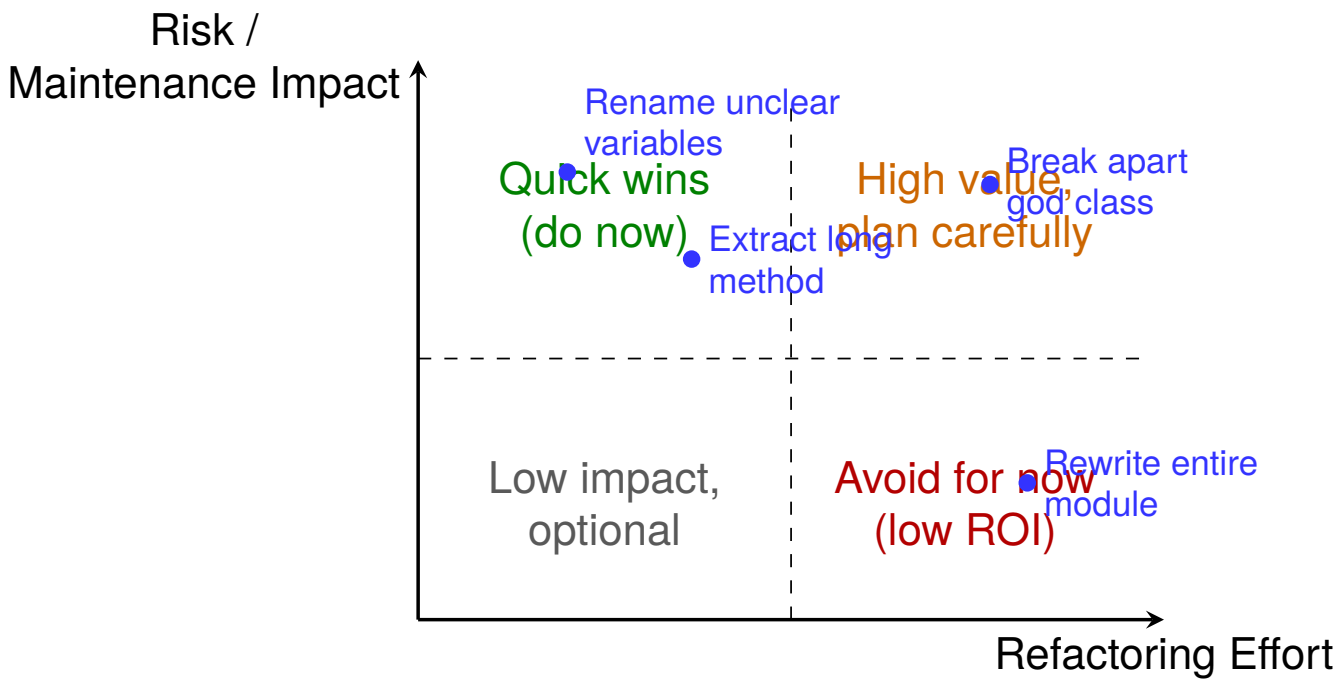
- **The God Class / Long Method:** A function or class that handles parsing, business logic, and database connections all at once. Violates the Single Responsibility Principle.
- **Duplicated Code:** The same logic exists in multiple places. If a bug is found, you have to remember to fix it everywhere.
- **Magic Numbers:** Hardcoded, unexplained numbers (e.g., `if (type == 1)` instead of `if (type == TYPE_RECON)`).
- **High Cyclomatic Complexity ("Arrow Code"):** Deeply nested `if/else` statements that push your code far to the right of the screen. Impossible to read and test.
- **Security Smells:** Using string concatenation for database queries (SQL Injection) instead of parameterized queries.

Common Code Smells

- **Long Method:** A function that spans multiple screens. It is doing too many things, making it impossible to unit test effectively.
- **Shotgun Surgery:** Every time you want to make a simple change (e.g., adding a new robot sensor), you have to make tiny edits to 15 different files.

Which Smells Should You Fix First?

Not all smells are equally urgent. Prioritize by **Impact** and **Effort**.



Interactive Task: Spot the Smells

Take 60 seconds. Read the code below. How many distinct ”Code Smells” can you find?

```
def do_stuff(x):
    if x == 1:
        # connect to DB
        db.execute("UPDATE robots SET status = 'active' WHERE id = 1")
    elif x == 2:
        # connect to DB
        db.execute("UPDATE robots SET status = 'active' WHERE id = 2")
    # ... repeats 10 more times ...
    print("Done checking. Now let's calculate battery...")
    # ... 50 more lines of battery calculation logic ...
```

Interactive Task: Spot the Smells

Take 60 seconds. Read the code below. How many distinct "Code Smells" can you find?

```
def do_stuff(x):
    if x == 1:
        # connect to DB
        db.execute("UPDATE robots SET status = 'active' WHERE id = 1")
    elif x == 2:
        # connect to DB
        db.execute("UPDATE robots SET status = 'active' WHERE id = 2")
    # ... repeats 10 more times ...
    print("Done checking. Now let's calculate battery...")
    # ... 50 more lines of battery calculation logic ...
```

The Smells:

- **Magic Numbers:** What do `1` and `2` mean?
- **Duplicated Code:** DB connection and update logic is copy-pasted.
- **Long Method / God Class:** Doing DB updates AND battery calculations in one function called `do_stuff`.

Smell → Refactoring Technique

Code Smell	Typical Refactoring	Expected Benefit
Long Method	Extract Method	Readability + testability
Duplicated Code	Pull Up Method / Helper Function	Single source of truth
God Class	Extract Class	Better modularity
Shotgun Surgery	Move Method / Introduce Facade	Localized future changes
Magic Numbers	Replace Magic Number with Symbolic Constant	Better intent and fewer mistakes

Key idea: Refactoring is not random cleanup; each smell has a targeted technique.

Agenda

The Solution: Refactoring

If Technical Debt is the disease, Refactoring is the cure.

Refactoring Definition

A change made to the internal structure of software to make it easier to understand and cheaper to modify **without changing its observable behavior**.

The Golden Rule of Refactoring: You cannot safely refactor code if you do not have an automated test suite (Week 8)! If you change the internal structure without tests, you are just blindly hoping you didn't break anything.

A Safe Refactoring Workflow (CI/CD-Aligned)

1. **Pin behavior first:** Add or update tests around the area you will change.
2. **Make one small refactor:** e.g., extract one method, rename one symbol.
3. **Run tests immediately:** Local tests + CI pipeline must stay green.
4. **Commit in tiny steps:** Small commits make rollbacks and reviews easier.
5. **Repeat:** Continuous improvement beats ”big bang” rewrites.

Rule

Red tests = stop refactoring. Fix behavior first, then continue cleanup.

Refactor or Rewrite?

"When should we just rewrite this from scratch?"

Prefer Refactor When...

- Behavior is mostly correct.
- Tests exist (or can be added quickly).
- Problems are localized to known hotspots.
- You can improve in small safe steps.

Consider Rewrite When...

- Core architecture blocks key requirements.
- The system is untestable and tightly coupled.
- Maintenance cost exceeds replacement cost.
- Scope is tightly controlled and phased.

Rule of thumb: If you cannot define safe, incremental steps, a rewrite is high risk.

Technical Debt Register (Team Practice)

Keep debt visible. Track it like any other engineering work item in Trello/GitHub.

Debt Item	Location	Interest Paid	Owner	Payoff Sprint	Risk
Duplicate validation	API service	Bugs fixed in 5 files	Team A	Sprint 7	Mediur
God class controller	Backend core	Slow onboarding + merge conflicts	Team B	Sprint 8	High
Hardcoded config values	Frontend config	Manual edits per deploy	Team C	Sprint 7	Low

Outcome: debt becomes planned work, not forgotten work.

How Do We Prove Refactoring Helped?

Refactoring should show measurable improvement, not just ”cleaner-looking” code.

- **Cyclomatic complexity:** Is decision complexity lower in changed modules?
- **Duplicated lines:** Did repeated logic decrease?
- **Test coverage (touched area):** Did confidence increase where code changed?
- **PR review/merge time:** Are changes easier to review and integrate?

Engineering mindset

Refactoring is an **investment**. We justify it with reduced delivery risk and lower future cost.

5-Minute Activity: Debt Triage

In pairs, rank each item by: **Severity**, **Interest Cost**, and **First Refactoring Step**.

1. 400-line function handling auth + DB + API calls.
2. Same SQL query copy-pasted in 6 files.
3. Hardcoded URLs and API keys in source code.
4. Feature change requires edits in 12 files.

5-Minute Activity: Debt Triage

In pairs, rank each item by: **Severity**, **Interest Cost**, and **First Refactoring Step**.

1. 400-line function handling auth + DB + API calls.
2. Same SQL query copy-pasted in 6 files.
3. Hardcoded URLs and API keys in source code.
4. Feature change requires edits in 12 files.

Debrief

Compare rankings with another pair. If rankings differ, justify using risk and effort.

Debt Triage: Example Ranking

Item	Severity	Interest	Rank	First Step
Hardcoded URLs and API keys	Critical	Very high	1	Move secrets/config out of code; rotate exposed keys
Feature change touches 12 files	High	Very high	2	Centralize the scattered behavior behind one abstraction
400-line auth + DB + API function	High	Very high	3	Add tests, then extract one responsibility at a time
Same SQL query in 6 files	Medium–High	High	4	Replace copy-paste with one shared query helper/service

Rule of thumb: Prioritize immediate security/operational risk first, then debt that makes every future change expensive.

Refactoring Example: Extract Method

The Smell: Long Method with unclear logic.

```
# BEFORE REFACTORING
def process_robot_telemetry(data):
    # Calculate battery percentage
    voltage = data['voltage']
    percentage = (voltage - 10.5) / (12.6 - 10.5) * 100
    if percentage > 100: percentage = 100
    if percentage < 0: percentage = 0

    # Save to database
    db.connect()
    db.execute(f"INSERT INTO telemetry (battery) VALUES ({percentage})")
    db.close()
```

Think-Pair-Share: How do we fix this?

The Bad Code:

```
def process_robot_telemetry(data):
    voltage = data['voltage']
    percentage = (voltage - 10.5) / (12.6 - 10.5) * 100
    if percentage > 100: percentage = 100
    if percentage < 0: percentage = 0

    db.connect()
    db.execute(f"INSERT INTO telemetry (battery) VALUES ({percentage})")
    db.close()
```

Task (2 minutes): Turn to the person next to you. If you had to refactor this using the ”Extract Method” technique, what would your new function signatures look like?

Think-Pair-Share: How do we fix this?

The Bad Code:

```
def process_robot_telemetry(data):
    voltage = data['voltage']
    percentage = (voltage - 10.5) / (12.6 - 10.5) * 100
    if percentage > 100: percentage = 100
    if percentage < 0: percentage = 0

    db.connect()
    db.execute(f"INSERT INTO telemetry (battery) VALUES ({percentage})")
    db.close()
```

Task (2 minutes): Turn to the person next to you. If you had to refactor this using the ”Extract Method” technique, what would your new function signatures look like?

(Let’s look at the solution on the next slide...)

Refactoring Example: Extract Method

The Fix: Extract the logic into smaller, testable, named methods.

```
# AFTER REFACTORING
def process_robot_telemetry(data):
    percentage = calculate_battery_percentage(data['voltage'])
    save_telemetry_to_db(percentage)

def calculate_battery_percentage(voltage):
    percentage = (voltage - 10.5) / (12.6 - 10.5) * 100
    return max(0, min(percentage, 100))

def save_telemetry_to_db(percentage):
    db.connect()
    db.execute("INSERT INTO telemetry (battery) VALUES (?)", (percentage,))
    db.close()
```

Behavior is identical. Readability, security (SQL injection fix), and testability are drastically improved.

Refactoring Example: Guard Clauses

The Smell: High Cyclomatic Complexity (Nested Ifs).

```
# BEFORE REFACTORING
def calculate_score(distance, battery):
    if distance > 0:
        if battery > 0:
            return (distance * 10) / battery
        else:
            return 0
    else:
        return 0
```

Refactoring Example: Guard Clauses

The Smell: High Cyclomatic Complexity (Nested Ifs).

```
# BEFORE REFACTORING
def calculate_score(distance, battery):
    if distance > 0:
        if battery > 0:
            return (distance * 10) / battery
        else:
            return 0
    else:
        return 0
```

The Fix: Use **Guard Clauses**. Invert the logic to check for the error/edge cases first, return early, and keep the ”happy path” totally flat at the bottom!

```
# AFTER REFACTORING
def calculate_score(distance, battery):
    # Guard clauses reject bad data immediately
    if distance <= 0 or battery <= 0:
        return 0

    # The "Happy Path" is no longer nested!
    return (distance * 10) / battery
```

Framework Smells

Sometimes code is perfectly valid Python, but it ignores the powerful tools provided by your chosen framework. This is a ”Framework Smell.”

Example in FastAPI: Manually parsing dictionaries instead of using Pydantic Models.

```python
# BAD: Manual parsing and validation
@app.post("/api/data")
def receive_data(data: dict):
    if "speed" not in data or type(data["speed"]) is not int:
        raise HTTPException(status_code=400)
    speed = data["speed"]
```

Framework Smells

Sometimes code is perfectly valid Python, but it ignores the powerful tools provided by your chosen framework. This is a "Framework Smell."

Example in FastAPI: Manually parsing dictionaries instead of using Pydantic Models.

```python
# BAD: Manual parsing and validation
@app.post("/api/data")
def receive_data(data: dict):
    if "speed" not in data or type(data["speed"]) is not int:
        raise HTTPException(status_code=400)
    speed = data["speed"]
```

```python
# GOOD: Let Pydantic do the heavy lifting!
class RobotData(BaseModel):
    speed: int

@app.post("/api/data")
def receive_data(data: RobotData):
    speed = data.speed # Guaranteed to be an integer!
```

Refactoring Pitfalls to Avoid

- **Big-bang refactor:** Huge changes with no checkpoints are high risk.
- **Refactoring without tests:** You cannot prove behavior stayed the same.
- **”Cosmetic only” changes:** Renaming everything without reducing complexity gives little value.
- **No documentation of intent:** If there is no issue/commit message, teams repeat the same mistakes.

Professional habit: Link each refactor to a concrete pain point (bug frequency, delivery delay, onboarding friction).

When should you refactor?

You don’t need to schedule a ”3-month refactoring sprint”. Refactoring should be a continuous, daily habit.

- **The Rule of Three:** The first time you do something, just do it. The second time, wince at the duplication, but do it. The third time you do the same thing, refactor it.
- **When adding a feature:** If the code is too messy to easily add a new feature, refactor the code first to make the addition easy.
- **The Boy Scout Rule:** ”Always leave the campground cleaner than you found it.” If you open a file to fix a bug and see a poorly named variable, rename it before you commit.

Agenda

Reading & Resources

- **Recommended Reading:** Sommerville, *Software Engineering 9th Edition*, Chapter 9 (Software Evolution).
- **Refactoring: Improving the Design of Existing Code** by Martin Fowler (The definitive guide to Code Smells and Refactoring techniques).
  `https://github.com/pratik0197/books/blob/master/software-dev/Refactoring%20%20Improving%20the%20Design%20of%20Existing%20Code.pdf`

Next Lecture: Release Management & Continuous Deployment

Any Questions?

fdelduchetto@lincoln.ac.uk